Universidad de Monterrey

Integración de aplicaciones computacionales

Tarea 4

Aplicación de microservicios en TKinter usando Python

Maestro: Dr. Raúl Morales Salcedo

Nombre: Margarita Concepción Cuervo Citalán #581771

Carrera: ITC, 9no Semestre

Día y hora del grupo de la clase: lunes y jueves 10:00h

12 de octubre de 2025.

Doy mi palabra de haber realizado esta actividad con integridad académica.

ÍNDICE

Introducción

En el desarrollo de sistemas basados en microservicios, la interfaz gráfica de usuario (GUI) suele implementarse comúnmente como una aplicación web usando HTML, CSS y JavaScript. Sin embargo, en este proyecto se decidió construir la interfaz utilizando Tkinter, el módulo estándar de Python para GUIs de escritorio.

El objetivo principal fue desarrollar una aplicación gráfica capaz de consumir de forma segura y eficiente los endpoints de un microservicio REST, incorporando autenticación JWT, manejo de libros y monitoreo de salud del servicio, todo desde una interfaz autónoma y fácil de usar.

Esta elección me permitió trabajar en un entorno local controlado, sin depender de navegadores ni tecnologías web externas, simplificando el flujo de pruebas y la integración directa con Python.

Desarrollo

Se empezó esta tarea por abrir el CMD de la computadora local y crear el directorio *temp* y el subdirectorio *PGUI* con los comandos **mkdir temp, cd temp, mkdir PGUI, cd PGUI.** Tras hacer esto el directorio se encontraba en **C:/temp/PGUI.**

Luego se fue a la VM de GCP y se seleccionó el microservicio de Libreria con JWT y Redis y se ejecutó este microservicio con los comandos **export FLASK_APP="main.py", python3 main.py.**

Una vez que el microservicio estaba corriendo se le pidio a la IA lo siguiente:
Crea un programa en Python con GUI usando Tkinter, quiero que se consuman los endpoints del siguiente microservicio.

1. El programa debe mostrar cada paso realizado en 1 microservicio.
2. Debe de mostrar en la GUI un log con la informacion de token JWT
3. Debe de tener los campos de usuario y contraseña para autenticarse
4. Debe de tener los campos para registrarse
5. Debe de tener en la GUI un semáforo que indique la salud del microservicio, rojo = no funciona, naranja = en proceso, y verde = saludable.
6. Debe de hacer uso del refresh token.
7. Implementa localstorage para guardar la ip, puerto, emdpoints y token JWT en la aplicacion

Diseña una GUI bien definida y fácil de usar, y en todo momento muestra el log de lo que sucede con el microservicio y los datos que recibe y manda.

el microservicio esta corriendo en http://127.0.0.1:5000
from flask import Flask, request, jsonify, Response, make_response
from flask_cors import CORS

```
from datetime import timedelta, datetime, timezone
from flask_jwt_extended import (
    JWTManager, create_access_token, create_refresh_token,
    jwt_required, get_jwt_identity, get_jwt
)
from flask_jwt_extended.utils import decode_token
from passlib.hash import pbkdf2_sha256
import MySQLdb
import xml.etree.ElementTree as ET
import redis
import os

# ========================
# App & Config
# ========================
app = Flask(__name__)

# JWT
app.config['JWT_SECRET_KEY'] = os.getenv('JWT_SECRET_KEY',
'super-secret-key-change-me')
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(days=30)
app.config['JWT_TOKEN_LOCATION'] = ['headers']
app.config['JWT_HEADER_NAME'] = 'Authorization'
app.config['JWT_HEADER_TYPE'] = 'Bearer'
jwt = JWTManager(app)

# CORS (abierto para pruebas)
CORS(
    app,
    resources={r"/*": {"origins": "*"}},
    methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    allow_headers=["Content-Type", "Authorization"],
    expose_headers=["Content-Type", "Authorization"],
    supports_credentials=False,
    send_wildcard=True,
    max_age=86400,
)

# MariaDB
DB_HOST = os.getenv("DB_HOST", "localhost")
DB_USER = os.getenv("DB_USER", "libro_user")
DB_PASS = os.getenv("DB_PASS", "666")
DB_NAME = os.getenv("DB_NAME", "Libros")
DB_CHARSET = "utf8"

def get_db():
    return MySQLdb.connect(
```

```
        host=DB_HOST, user=DB_USER, passwd=DB_PASS,
        db=DB_NAME, charset=DB_CHARSET
    )

# Redis
REDIS_HOST = os.getenv("REDIS_HOST", "127.0.0.1")
REDIS_PORT = int(os.getenv("REDIS_PORT", "6379"))
REDIS_DB   = int(os.getenv("REDIS_DB", "0"))
r = redis.Redis(host=REDIS_HOST, port=REDIS_PORT, db=REDIS_DB,
decode_responses=True)

# =========================
# Helpers: JWT + Redis
# =========================
def _now_ts():
    return int(datetime.now(timezone.utc).timestamp())

def _ttl_from_exp(exp_unix: int) -> int:
    ttl = exp_unix - _now_ts()
    return ttl if ttl > 0 else 1

def _allow_key(token_type: str, jti: str) -> str:
    return f"allow:{token_type}:{jti}"

def _block_key(jti: str) -> str:
    return f"block:{jti}"

def store_in_allowlist(encoded_token: str, token_type: str, username: str):
    decoded = decode_token(encoded_token)
    jti = decoded['jti']
    exp = decoded['exp']
    ttl = _ttl_from_exp(exp)
    key = _allow_key(token_type, jti)
    r.hset(key, mapping={"username": username, "type": token_type, "exp": str(exp)})
    r.expire(key, ttl)
    return jti

def is_in_allowlist(jti: str) -> bool:
    # válido si existe en allowlist (access o refresh)
    return r.exists(_allow_key("access", jti)) == 1 or r.exists(_allow_key("refresh", jti)) == 1

def is_revoked(jti: str) -> bool:
    return r.exists(_block_key(jti)) == 1

def revoke_jti(jti: str, exp: int):
    ttl = _ttl_from_exp(exp)
    r.setex(_block_key(jti), ttl, "1")
    # opcional: limpiar allowlist
```

```
    r.delete(_allow_key("access", jti))
    r.delete(_allow_key("refresh", jti))

@jwt.token_in_blocklist_loader
def token_check_revoked(jwt_header, jwt_payload):
    # Política estricta: si NO está en allowlist -> inválido
    jti = jwt_payload["jti"]
    return is_revoked(jti) or (not is_in_allowlist(jti))

# Preflight (CORS)
@app.before_request
def handle_preflight():
    if request.method == "OPTIONS":
        resp = make_response()
        resp.headers["Access-Control-Allow-Origin"] = request.headers.get("Origin", "*")
        resp.headers["Vary"] = "Origin"
        resp.headers["Access-Control-Allow-Headers"] = "Content-Type, Authorization"
        resp.headers["Access-Control-Allow-Methods"] = "GET,POST,PUT,DELETE,OPTIONS"
        return resp, 200

@app.after_request
def add_cors_headers(resp):
    resp.headers["Access-Control-Allow-Origin"] = request.headers.get("Origin", "*")
    resp.headers["Vary"] = "Origin"
    resp.headers["Access-Control-Allow-Headers"] = "Content-Type, Authorization"
    resp.headers["Access-Control-Allow-Methods"] = "GET,POST,PUT,DELETE,OPTIONS"
    return resp

# ========================
# XML helpers
# ========================
def books_to_xml(rows):
    catalog = ET.Element('catalog')
    for b in rows:
        book_elem = ET.Element('book', isbn=b.get('isbn', ''))
        title = ET.SubElement(book_elem, 'title'); title.text = b.get('titulo', 'Desconocido')
        # soporta múltiples autores (si hacen join con GROUP_CONCAT)
        authors_field = b.get('authors') or b.get('autor') or ''
        authors = authors_field.split(',') if authors_field else []
        if not authors:
            ET.SubElement(book_elem, 'author').text = 'Desconocido'
        else:
            for a in authors:
                ET.SubElement(book_elem, 'author').text = a.strip()

        year = ET.SubElement(book_elem, 'year'); year.text = str(b.get('anio_publicacion', '0'))
        genre = ET.SubElement(book_elem, 'genre'); genre.text = b.get('genre_name',
b.get('genero', 'Desconocido'))
```

```
        price = ET.SubElement(book_elem, 'price'); price.text = str(b.get('price', '0.0'))
        stock = ET.SubElement(book_elem, 'stock'); stock.text = str(b.get('stock', '0'))
        fmt = ET.SubElement(book_elem, 'format'); fmt.text = b.get('formato', 'Desconocido')
        catalog.append(book_elem)
    return ET.tostring(catalog, encoding='utf-8', xml_declaration=True)

# ========================
# AUTH
# ========================

# POST /auth/register
@app.route('/auth/register', methods=['POST'])
def register():
    data = request.get_json(silent=True) or {}
    username = data.get('username'); password = data.get('password')
    if not username or not password:
        return jsonify({"msg": "username y password son requeridos"}), 400
    try:
        conn = get_db(); cur = conn.cursor()
        cur.execute("SELECT 1 FROM Users WHERE username=%s", (username,))
        if cur.fetchone():
            return jsonify({"msg": "Usuario ya existe"}), 409
        cur.execute("INSERT INTO Users (username, password_hash) VALUES (%s,%s)",
                    (username, pbkdf2_sha256.hash(password)))
        conn.commit()
        return jsonify({"msg": "Usuario creado"}), 201
    except Exception as e:
        return jsonify({"msg": "DB error", "error": str(e)}), 500
    finally:
        try: conn.close()
        except: pass

# POST /auth/login
@app.route('/auth/login', methods=['POST'])
def login():
    data = request.get_json(silent=True) or {}
    username = data.get('username'); password = data.get('password')
    if not username or not password:
        return jsonify({"msg": "username y password son requeridos"}), 400
    try:
        conn = get_db(); cur = conn.cursor(MySQLdb.cursors.DictCursor)
        cur.execute("SELECT password_hash FROM Users WHERE username=%s",
(username,))
        row = cur.fetchone()
        if not row or not pbkdf2_sha256.verify(password, row['password_hash']):
            return jsonify({"msg": "Credenciales inválidas"}), 401

        access  = create_access_token(identity=username)
```

```
        refresh = create_refresh_token(identity=username)
        jti_a   = store_in_allowlist(access,  "access",  username)
        jti_r   = store_in_allowlist(refresh, "refresh", username)
        return jsonify(access_token=access, refresh_token=refresh,
                       jti_access=jti_a, jti_refresh=jti_r), 200
    except Exception as e:
        return jsonify({"msg":"DB/Auth error","error":str(e)}), 500
    finally:
        try: conn.close()
        except: pass

# POST /auth/refresh
@app.route('/auth/refresh', methods=['POST'])
@jwt_required(refresh=True)
def refresh_token():
    user = get_jwt_identity()
    new_access = create_access_token(identity=user)
    jti_a = store_in_allowlist(new_access, "access", user)
    return jsonify(access_token=new_access, jti_access=jti_a), 200

# POST /auth/logout (revoca el token access actual)
@app.route('/auth/logout', methods=['POST'])
@jwt_required()
def logout():
    claims = get_jwt()
    revoke_jti(claims['jti'], claims['exp'])
    return jsonify({"msg":"logout ok","revoked_jti":claims['jti']}), 200

# ========================
# BOOKS (TODOS protegidos)
# ========================

# GET /api/books → XML con todos
@app.route('/api/books', methods=['GET'])
@jwt_required()
def books_all():
    try:
        conn = get_db(); cur = conn.cursor(MySQLdb.cursors.DictCursor)
        cur.execute("""
            SELECT l.isbn, l.titulo, l.anio_publicacion, l.price, l.stock, l.formato,
                   GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                   c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
            GROUP BY l.isbn
        """)
        rows = cur.fetchall()
```

```
        xml = books_to_xml(rows)
        return Response(xml, mimetype='application/xml', status=200)
    except Exception as e:
        err = f"<?xml version='1.0' encoding='UTF-8'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
        try: conn.close()
        except: pass

# GET /api/books/ISBN → ?isbn=...
@app.route('/api/books/ISBN', methods=['GET'], strict_slashes=False)
@jwt_required()
def books_by_isbn():
    isbn = request.args.get('isbn', '').strip()
    if not isbn:
        return Response("<?xml version='1.0'?><error>isbn requerido</error>",
                        mimetype='application/xml', status=400)
    try:
        conn = get_db(); cur = conn.cursor(MySQLdb.cursors.DictCursor)
        cur.execute("""
            SELECT l.isbn, l.titulo, l.anio_publicacion, l.price, l.stock, l.formato,
                   GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                   c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
            WHERE l.isbn = %s
            GROUP BY l.isbn
        """, (isbn,))
        rows = cur.fetchall()
        xml = books_to_xml(rows)
        return Response(xml, mimetype='application/xml', status=200)
    except Exception as e:
        err = f"<?xml version='1.0'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
        try: conn.close()
        except: pass

# GET /api/books/format/ → ?format=...
@app.route('/api/books/format/', methods=['GET'], strict_slashes=False)
@jwt_required()
def books_by_format():
    fmt = request.args.get('format', '').strip()
    if not fmt:
        return Response("<?xml version='1.0'?><error>format requerido</error>",
                        mimetype='application/xml', status=400)
    try:
```

```
        conn = get_db(); cur = conn.cursor(MySQLdb.cursors.DictCursor)
        cur.execute("""
            SELECT l.isbn, l.titulo, l.anio_publicacion, l.price, l.stock, l.formato,
                   GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                   c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
            WHERE l.formato = %s
            GROUP BY l.isbn
        """, (fmt,))
        rows = cur.fetchall()
        xml = books_to_xml(rows)
        return Response(xml, mimetype='application/xml', status=200)
    except Exception as e:
        err = f"<?xml version='1.0'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
        try: conn.close()
        except: pass

# GET /api/books/autor/ → ?name=...
@app.route('/api/books/autor/', methods=['GET'], strict_slashes=False)
@jwt_required()
def books_by_autor():
    name = request.args.get('name', '').strip()
    if not name:
        return Response("<?xml version='1.0'?><error>name requerido</error>",
                        mimetype='application/xml', status=400)
    try:
        conn = get_db(); cur = conn.cursor(MySQLdb.cursors.DictCursor)
        cur.execute("""
            SELECT l.isbn, l.titulo, l.anio_publicacion, l.price, l.stock, l.formato,
                   GROUP_CONCAT(a.nombre SEPARATOR ',') AS authors,
                   c.nombre AS genre_name
            FROM Libro l
            LEFT JOIN Autor a ON l.id_autor = a.id_autor
            LEFT JOIN Categoria c ON l.id_categoria = c.id_categoria
            WHERE a.nombre LIKE %s
            GROUP BY l.isbn
        """, (f"%{name}%",))
        rows = cur.fetchall()
        xml = books_to_xml(rows)
        return Response(xml, mimetype='application/xml', status=200)
    except Exception as e:
        err = f"<?xml version='1.0'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
```

```
        try: conn.close()
        except: pass

# POST /api/books/create
@app.route('/api/books/create', methods=['POST'], strict_slashes=False)
@jwt_required()
def book_create():
    data = request.get_json(silent=True) or {}
    req = ['isbn','titulo','id_autor','id_categoria','id_editorial',
           'anio_publicacion','price','stock','formato']
    if any(k not in data for k in req):
        return Response("<?xml version='1.0'?><error>faltan campos</error>",
                        mimetype='application/xml', status=400)
    try:
        conn = get_db(); cur = conn.cursor()
        cur.execute("""
            INSERT INTO Libro (isbn, titulo, id_autor, id_categoria, id_editorial,
                               anio_publicacion, price, stock, formato)
            VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)
        """, (data['isbn'], data['titulo'], data['id_autor'], data['id_categoria'],
              data['id_editorial'], data['anio_publicacion'], data['price'],
              data['stock'], data['formato']))
        conn.commit()
        return Response("<?xml version='1.0'?><ok>inserted</ok>",
                        mimetype='application/xml', status=201)
    except Exception as e:
        err = f"<?xml version='1.0'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
        try: conn.close()
        except: pass

# PUT /api/books/update
@app.route('/api/books/update', methods=['PUT'], strict_slashes=False)
@jwt_required()
def book_update():
    data = request.get_json(silent=True) or {}
    isbn = data.get('isbn')
    if not isbn:
        return Response("<?xml version='1.0'?><error>isbn requerido</error>",
                        mimetype='application/xml', status=400)
    fields = {k:v for k,v in data.items() if k in [
        'titulo','id_autor','id_categoria','id_editorial',
        'anio_publicacion','price','stock','formato'
    ] and v not in (None, '')}
    if not fields:
        return Response("<?xml version='1.0'?><error>nada que actualizar</error>",
                        mimetype='application/xml', status=400)
```

```
    try:
        conn = get_db(); cur = conn.cursor()
        sets = ", ".join([f"{k}=%s" for k in fields.keys()])
        params = list(fields.values()) + [isbn]
        cur.execute(f"UPDATE Libro SET {sets} WHERE isbn=%s", params)
        conn.commit()
        return Response("<?xml version='1.0'?><ok>updated</ok>",
                        mimetype='application/xml', status=200)
    except Exception as e:
        err = f"<?xml version='1.0'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
        try: conn.close()
        except: pass

# DELETE /api/books/delete → ?isbn=...
@app.route('/api/books/delete', methods=['DELETE'], strict_slashes=False)
@jwt_required()
def book_delete():
    isbn = request.args.get('isbn','').strip()
    if not isbn:
        return Response("<?xml version='1.0'?><error>isbn requerido</error>",
                        mimetype='application/xml', status=400)
    try:
        conn = get_db(); cur = conn.cursor()
        cur.execute("DELETE FROM Libro WHERE isbn=%s", (isbn,))
        conn.commit()
        return Response("<?xml version='1.0'?><ok>deleted</ok>",
                        mimetype='application/xml', status=200)
    except Exception as e:
        err = f"<?xml version='1.0'?><error>{str(e)}</error>"
        return Response(err, mimetype='application/xml', status=500)
    finally:
        try: conn.close()
        except: pass

# Salud
@app.route('/ping', methods=['GET'])
def ping():
    return jsonify(msg="pong"), 200

if __name__ == '__main__':
    # Escucha en todas las interfaces
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Tras ejecutar este prompt y pedirle que arreglara el log y el despliegue de la lista de libros una vez que el usuario estuviera autenticado la IA me dio el código del archivo **gui_jwt_client.py:**

Python

```python
import json
import os
import threading
import time
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
from tkinter.scrolledtext import ScrolledText
from urllib.parse import urljoin, urlencode

import requests
import jwt  # PyJWT
import xml.etree.ElementTree as ET

# =========================
# "LocalStorage" en archivo
# =========================
CONFIG_PATH = os.path.join(os.path.expanduser("~"), ".jwt_books_gui_config.json")
LOG_PATH    = os.path.join(os.path.expanduser("~"), ".jwt_books_gui.log")
MAX_LOG_FILE_BYTES = 1_000_000  # 1MB rotación simple

DEFAULT_CFG = {
    "ip": "127.0.0.1",
    "port": 5000,
    "endpoints": {
        "register": "/auth/register",
        "login": "/auth/login",
        "refresh": "/auth/refresh",
        "logout": "/auth/logout",
        "ping": "/ping",
        "books_all": "/api/books",
        "books_by_isbn": "/api/books/ISBN",
        "books_by_format": "/api/books/format/",
        "books_by_autor": "/api/books/autor/",
        "book_create": "/api/books/create",
        "book_update": "/api/books/update",
        "book_delete": "/api/books/delete",
    },
    "access_token": "",
    "refresh_token": ""
}

def load_config():
    if os.path.exists(CONFIG_PATH):
        try:
            with open(CONFIG_PATH, "r", encoding="utf-8") as f:
                data = json.load(f)
                cfg = DEFAULT_CFG.copy()
                cfg.update(data)
                if "endpoints" in data:
                    tmp = DEFAULT_CFG["endpoints"].copy()
                    tmp.update(data["endpoints"])
                    cfg["endpoints"] = tmp
```

```
                return cfg
        except Exception:
            pass
    return DEFAULT_CFG.copy()

def save_config(cfg):
    try:
        with open(CONFIG_PATH, "w", encoding="utf-8") as f:
            json.dump(cfg, f, indent=2, ensure_ascii=False)
        return True
    except Exception:
        return False

def _rotate_log_file_if_needed():
    try:
        if os.path.exists(LOG_PATH) and os.path.getsize(LOG_PATH) >
MAX_LOG_FILE_BYTES:
            base = LOG_PATH + ".1"
            if os.path.exists(base):
                os.remove(base)
            os.replace(LOG_PATH, base)
    except Exception:
        pass

def _append_file_log(line: str):
    try:
        _rotate_log_file_if_needed()
        with open(LOG_PATH, "a", encoding="utf-8") as f:
            f.write(line + "\n")
    except Exception:
        pass

# =========================
# Cliente HTTP con refresh
# =========================
class ApiClient:
    def __init__(self, cfg, log_fn, log_http_detail_fn):
        self.cfg = cfg
        self.log = log_fn  # texto plano con niveles
        self.log_http_detail = log_http_detail_fn  # para volcar REQUEST/RESPONSE
        self.session = requests.Session()

        # última transacción HTTP (para "Copiar cURL")
        self.last_request = None   # dict
        self.last_response = None  # dict

    def base_url(self):
        return f"http://{self.cfg['ip']}:{self.cfg['port']}/"

    def ep(self, key):
        return urljoin(self.base_url(), self.cfg["endpoints"][key].lstrip("/"))

    # ---------- Helpers ----------
    def _auth_headers(self, use_refresh=False):
```

```
        token = (self.cfg.get("refresh_token") if use_refresh else
self.cfg.get("access_token")) or ""
        return {"Authorization": f"Bearer {token}"} if token else {}

    def _record_http(self, method, url, req_headers, req_body, resp):
        # guarda info para cURL
        try:
            self.last_request = {
                "method": method,
                "url": url,
                "headers": dict(req_headers) if req_headers else {},
                "body": req_body if req_body is not None else None
            }
            self.last_response = {
                "status": resp.status_code,
                "headers": dict(resp.headers),
                "text": resp.text
            }
        except Exception:
            pass

    def _raw_send(self, method, url, **kwargs):
        """Envia request sin reintentos, registrando REQUEST/RESPONSE detallado."""
        req_headers = kwargs.get("headers") or {}
        req_body    = None
        if "json" in kwargs and kwargs["json"] is not None:
            req_body = kwargs["json"]
        elif "data" in kwargs and kwargs["data"] is not None:
            req_body = kwargs["data"]

        self.log("INFO", "HTTP", f"{method} {url}")
        try:
            resp = self.session.request(method, url, timeout=15, **kwargs)
            # log detallado
            self._record_http(method, url, req_headers, req_body, resp)
            self.log_http_detail(method, url, req_headers, req_body, resp)
            return resp
        except requests.RequestException as e:
            self.log("ERROR", "HTTP", f"Petición falló: {e}")
            # registra aunque falle
            self._record_http(method, url, req_headers, req_body, getattr(e,
"response", None) or requests.Response())
            raise

    def _request(self, method, url, **kwargs):
        """
        Envoltorio con:
        - Bearer Access
        - Si 401 y tenemos refresh, intenta refrescar y reintentar una vez
        - Log detallado REQUEST/RESPONSE
        """
        headers = kwargs.pop("headers", {})
        headers.update(self._auth_headers(use_refresh=False))
        kwargs["headers"] = headers
```

```
        try:
            resp = self._raw_send(method, url, **kwargs)
        except requests.RequestException:
            raise

        if resp.status_code == 401 and self.cfg.get("refresh_token"):
            self.log("WARN", "AUTH", "401 recibido, intentando refresh token…")
            if self.refresh_access_token():
                headers = kwargs.get("headers", {})
                headers.update(self._auth_headers(use_refresh=False))
                kwargs["headers"] = headers
                self.log("INFO", "AUTH", "Reintentando con nuevo access token…")
                resp = self._raw_send(method, url, **kwargs)

        return resp

    # ---- AUTH FLOWS ----
    def register(self, username, password):
        url = self.ep("register")
        payload = {"username": username, "password": password}
        return self._raw_send("POST", url, json=payload)

    def login(self, username, password):
        url = self.ep("login")
        payload = {"username": username, "password": password}
        resp = self._raw_send("POST", url, json=payload)
        if resp.ok:
            try:
                data = resp.json()
            except Exception:
                data = {}
            self.cfg["access_token"] = data.get("access_token", "")
            self.cfg["refresh_token"] = data.get("refresh_token", "")
            save_config(self.cfg)
            self.log_tokens()
        return resp

    def refresh_access_token(self):
        url = self.ep("refresh")
        headers = self._auth_headers(use_refresh=True)
        try:
            resp = self._raw_send("POST", url, headers=headers)
            if resp.ok:
                try:
                    data = resp.json()
                except Exception:
                    data = {}
                self.cfg["access_token"] = data.get("access_token", "")
                save_config(self.cfg)
                self.log_tokens()
                return True
            return False
        except requests.RequestException as e:
```

```
            self.log("ERROR", "AUTH", f"Refresh falló: {e}")
            return False

    def logout(self):
        url = self.ep("logout")
        resp = self._request("POST", url)
        if resp.ok:
            self.cfg["access_token"] = ""
            self.cfg["refresh_token"] = ""
            save_config(self.cfg)
        return resp

    def log_tokens(self):
        at = self.cfg.get("access_token", "")
        rt = self.cfg.get("refresh_token", "")
        def decode_token(name, tok):
            if not tok:
                self.log("INFO", "JWT", f"{name}: <vacío>")
                return
            try:
                header = jwt.get_unverified_header(tok)
                payload = jwt.decode(tok, options={"verify_signature": False})
                self.log("DEBUG", "JWT", f"{name} header: {header}")
                self.log("INFO",  "JWT", f"{name} sub: {payload.get('sub') or
payload.get('identity')}")
                if "exp" in payload:
                    now = int(time.time())
                    ttl = payload["exp"] - now
                    self.log("INFO", "JWT", f"{name} exp en {ttl} s")
            except Exception as e:
                self.log("ERROR", "JWT", f"{name} decode error: {e}")

        decode_token("ACCESS", at)
        decode_token("REFRESH", rt)

    # ---- HEALTH ----
    def ping(self):
        url = self.ep("ping")
        try:
            resp = self._raw_send("GET", url)
            return resp.ok
        except requests.RequestException:
            return False

    # ---- BOOKS ----
    def books_all(self):
        url = self.ep("books_all")
        return self._request("GET", url)

    def books_by_isbn(self, isbn):
        url = self.ep("books_by_isbn")
        url = f"{url}?{urlencode({'isbn': isbn})}"
        return self._request("GET", url)
```

```
    def books_by_format(self, fmt):
        url = self.ep("books_by_format")
        url = f"{url}?{urlencode({'format': fmt})}"
        return self._request("GET", url)

    def books_by_autor(self, name):
        url = self.ep("books_by_autor")
        url = f"{url}?{urlencode({'name': name})}"
        return self._request("GET", url)

    def book_create(self, data):
        url = self.ep("book_create")
        return self._request("POST", url, json=data)

    def book_update(self, data):
        url = self.ep("book_update")
        return self._request("PUT", url, json=data)

    def book_delete(self, isbn):
        url = self.ep("book_delete")
        url = f"{url}?{urlencode({'isbn': isbn})}"
        return self._request("DELETE", url)

# =========================
# GUI
# =========================
class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Cliente JWT + Libros (Tkinter)")
        self.geometry("1180x820")
        self.minsize(1040, 720)

        self.cfg = load_config()
        self.verbose_level = tk.StringVar(value="INFO")  # INFO/DEBUG
        self.show_headers  = tk.BooleanVar(value=True)
        self.show_body     = tk.BooleanVar(value=True)

        self.client = ApiClient(self.cfg, self._log_line, self._log_http_detail)

        self.auth_ready = bool(self.cfg.get("access_token"))

        self._build_ui()
        self._start_health_monitor()

    # ---------- UI ----------
    def _build_ui(self):
        # Top bar
        top = ttk.Frame(self); top.pack(fill="x", padx=10, pady=8)
        ttk.Label(top, text="Semáforo salud:").pack(side="left")
        self.sem_canvas = tk.Canvas(top, width=22, height=22, highlightthickness=0)
        self.sem_canvas.pack(side="left", padx=6)
        self.sem_light = self.sem_canvas.create_oval(2, 2, 20, 20, fill="#b22222",
outline="")
```

```
        ttk.Label(top, text="Base URL:").pack(side="left", padx=(15, 4))
        self.base_url_var = tk.StringVar(value=self.client.base_url())
        ttk.Entry(top, textvariable=self.base_url_var, width=44,
state="readonly").pack(side="left")
        ttk.Button(top, text="Ping", command=self._ping_now).pack(side="left", padx=8)

        # Notebook
        nb = ttk.Notebook(self); nb.pack(fill="both", expand=True, padx=10,
pady=(0,10))
        self.tab_auth  = ttk.Frame(nb); nb.add(self.tab_auth,  text="Auth")
        self.tab_books = ttk.Frame(nb); nb.add(self.tab_books, text="Libros")
        self.tab_config= ttk.Frame(nb); nb.add(self.tab_config, text="Config")

        self._build_auth_tab()
        self._build_books_tab()
        self._build_config_tab()

        # Log + controles
        log_outer = ttk.Frame(self); log_outer.pack(fill="both", expand=True, padx=10,
pady=(0,10))
        controls = ttk.Frame(log_outer); controls.pack(fill="x", pady=(0,4))

        ttk.Label(controls, text="Nivel:").pack(side="left")
        cb = ttk.Combobox(controls, values=["DEBUG","INFO","WARN","ERROR"],
textvariable=self.verbose_level, width=7, state="readonly")
        cb.pack(side="left", padx=(4,10))

        ttk.Checkbutton(controls, text="Mostrar headers",
variable=self.show_headers).pack(side="left")
        ttk.Checkbutton(controls, text="Mostrar cuerpo",
variable=self.show_body).pack(side="left", padx=(8,10))

        ttk.Button(controls, text="Copiar cURL último request",
command=self._copy_curl).pack(side="left", padx=6)
        ttk.Button(controls, text="Guardar log…",
command=self._save_log_as).pack(side="left", padx=6)
        ttk.Button(controls, text="Limpiar",
command=self._clear_log).pack(side="left", padx=6)

        log_frame = ttk.LabelFrame(log_outer, text="Log en vivo
(requests/responses/JWT)")
        log_frame.pack(fill="both", expand=True)
        self.log_text = ScrolledText(log_frame, height=14, wrap="word")
        self.log_text.pack(fill="both", expand=True)

        # Tags de color
        self._init_log_tags()

        self._update_token_fields()
        self._set_books_enabled(self.auth_ready)

    def _init_log_tags(self):
        # Colores por categoría
        self.log_text.tag_configure("TS", foreground="#888888")
```

```
        self.log_text.tag_configure("INFO", foreground="#1b5e20")   # verde oscuro
        self.log_text.tag_configure("DEBUG", foreground="#0d47a1")  # azul
        self.log_text.tag_configure("WARN", foreground="#e65100")   # naranja
        self.log_text.tag_configure("ERROR", foreground="#b71c1c", underline=True)  #
rojo
        self.log_text.tag_configure("HTTP", foreground="#4a148c")   # morado
        self.log_text.tag_configure("JWT", foreground="#006064")    # teal
        self.log_text.tag_configure("BOOKS", foreground="#3e2723")  # café
        self.log_text.tag_configure("HEALTH", foreground="#263238") # gris azulado
        self.log_text.tag_configure("CFG", foreground="#1a237e")    # azul indigo
        self.log_text.tag_configure("HEAD", foreground="#455a64")   # gris headers
        self.log_text.tag_configure("BODY", foreground="#37474f")   # gris texto

    # ---------- Auth tab ----------
    def _build_auth_tab(self):
        f = self.tab_auth
        lf = ttk.LabelFrame(f, text="Login"); lf.pack(fill="x", padx=10, pady=10)
        self.login_user = tk.StringVar(); self.login_pass = tk.StringVar()
        row = ttk.Frame(lf); row.pack(fill="x", pady=4)
        ttk.Label(row, text="Usuario: ").pack(side="left")
        ttk.Entry(row, textvariable=self.login_user, width=30).pack(side="left",
padx=4)
        row = ttk.Frame(lf); row.pack(fill="x", pady=4)
        ttk.Label(row, text="Contraseña: ").pack(side="left")
        ttk.Entry(row, textvariable=self.login_pass, width=30,
show="*").pack(side="left", padx=4)
        btnrow = ttk.Frame(lf); btnrow.pack(fill="x", pady=6)
        self.btn_login   = ttk.Button(btnrow, text="Login", command=self._do_login);
self.btn_login.pack(side="left")
        self.btn_refresh = ttk.Button(btnrow, text="Refresh Access",
command=self._do_refresh); self.btn_refresh.pack(side="left", padx=8)
        self.btn_logout  = ttk.Button(btnrow, text="Logout", command=self._do_logout);
self.btn_logout.pack(side="left")

        rf = ttk.LabelFrame(f, text="Registro"); rf.pack(fill="x", padx=10, pady=10)
        self.reg_user = tk.StringVar(); self.reg_pass = tk.StringVar()
        row = ttk.Frame(rf); row.pack(fill="x", pady=4)
        ttk.Label(row, text="Usuario: ").pack(side="left")
        ttk.Entry(row, textvariable=self.reg_user, width=30).pack(side="left", padx=4)
        row = ttk.Frame(rf); row.pack(fill="x", pady=4)
        ttk.Label(row, text="Contraseña: ").pack(side="left")
        ttk.Entry(row, textvariable=self.reg_pass, width=30,
show="*").pack(side="left", padx=4)
        btnrow = ttk.Frame(rf); btnrow.pack(fill="x", pady=6)
        self.btn_register = ttk.Button(btnrow, text="Registrar",
command=self._do_register); self.btn_register.pack(side="left")

        tf = ttk.LabelFrame(f, text="Tokens actuales (solo lectura)");
tf.pack(fill="x", padx=10, pady=10)
        self.at_var = tk.StringVar(value=""); self.rt_var = tk.StringVar(value="")
        row = ttk.Frame(tf); row.pack(fill="x", pady=4)
        ttk.Label(row, text="Access:").pack(side="left")
        ttk.Entry(row, textvariable=self.at_var, width=95,
state="readonly").pack(side="left", padx=4)
```

```
        row = ttk.Frame(tf); row.pack(fill="x", pady=4)
        ttk.Label(row, text="Refresh:").pack(side="left")
        ttk.Entry(row, textvariable=self.rt_var, width=95,
state="readonly").pack(side="left", padx=4)
        self.btn_decode = ttk.Button(tf, text="Decodificar y loguear JWT",
command=self._log_tokens)
        self.btn_decode.pack(side="left", padx=10, pady=6)

    # ---------- Books tab ----------
    def _build_books_tab(self):
        f = self.tab_books
        qf = ttk.LabelFrame(f, text="Consultas"); qf.pack(fill="x", padx=10, pady=10)
        self.btn_books_all = ttk.Button(qf, text="GET /api/books (todos)",
command=self._books_all)
        self.btn_books_all.pack(side="left", padx=6, pady=6)
        ttk.Label(qf, text="ISBN:").pack(side="left")
        self.q_isbn = tk.StringVar()
        ttk.Entry(qf, textvariable=self.q_isbn, width=20).pack(side="left", padx=4)
        self.btn_books_isbn = ttk.Button(qf, text="GET por ISBN",
command=self._books_by_isbn)
        self.btn_books_isbn.pack(side="left", padx=6)
        ttk.Label(qf, text="Format:").pack(side="left", padx=(12,0))
        self.q_fmt = tk.StringVar()
        ttk.Entry(qf, textvariable=self.q_fmt, width=15).pack(side="left", padx=4)
        self.btn_books_fmt = ttk.Button(qf, text="GET por format",
command=self._books_by_format)
        self.btn_books_fmt.pack(side="left", padx=6)
        ttk.Label(qf, text="Autor:").pack(side="left", padx=(12,0))
        self.q_autor = tk.StringVar()
        ttk.Entry(qf, textvariable=self.q_autor, width=20).pack(side="left", padx=4)
        self.btn_books_autor = ttk.Button(qf, text="GET por autor",
command=self._books_by_autor)
        self.btn_books_autor.pack(side="left", padx=6)

        # Tabla
        table_frame = ttk.LabelFrame(f, text="Resultado")
        table_frame.pack(fill="both", expand=True, padx=10, pady=(0,10))
        cols = ("isbn","title","authors","year","genre","price","stock","format")
        self.tree = ttk.Treeview(table_frame, columns=cols, show="headings",
height=12)
        headers =
{"isbn":"ISBN","title":"Título","authors":"Autores","year":"Año","genre":"Género","pri
ce":"Precio","stock":"Stock","format":"Formato"}
        widths  =
{"isbn":120,"title":240,"authors":220,"year":70,"genre":120,"price":80,"stock":70,"for
mat":90}
        for c in cols:
            self.tree.heading(c, text=headers[c]); self.tree.column(c,
width=widths[c], anchor="w")
        self.tree.pack(fill="both", expand=True, side="left")
        yscroll = ttk.Scrollbar(table_frame, orient="vertical",
command=self.tree.yview)
        yscroll.pack(side="right", fill="y");
self.tree.configure(yscrollcommand=yscroll.set)
```

```
        # CRUD
        mf = ttk.LabelFrame(f, text="Crear/Actualizar/Borrar Libro")
        mf.pack(fill="x", padx=10, pady=10)
        self.b_isbn  = tk.StringVar(); self.b_titulo= tk.StringVar()
        self.b_id_autor = tk.StringVar(); self.b_id_categoria = tk.StringVar()
        self.b_id_editorial = tk.StringVar(); self.b_anio = tk.StringVar()
        self.b_price= tk.StringVar(); self.b_stock= tk.StringVar(); self.b_formato =
tk.StringVar()
        def row(frm, label, var, width=18):
            r = ttk.Frame(frm); r.pack(side="left", padx=6, pady=4)
            ttk.Label(r, text=label).pack(anchor="w")
            ttk.Entry(r, textvariable=var, width=width).pack(anchor="w")
        row(mf, "isbn*", self.b_isbn); row(mf, "titulo", self.b_titulo, 24)
        row(mf, "id_autor", self.b_id_autor); row(mf, "id_categoria",
self.b_id_categoria)
        row(mf, "id_editorial", self.b_id_editorial); row(mf, "anio_publicacion",
self.b_anio)
        row(mf, "price", self.b_price); row(mf, "stock", self.b_stock); row(mf,
"formato", self.b_formato)
        btns = ttk.Frame(f); btns.pack(fill="x", padx=10, pady=6)
        self.btn_create = ttk.Button(btns, text="POST create",
command=self._book_create); self.btn_create.pack(side="left")
        self.btn_update = ttk.Button(btns, text="PUT update",
command=self._book_update); self.btn_update.pack(side="left", padx=10)
        self.btn_delete = ttk.Button(btns, text="DELETE por ISBN",
command=self._book_delete); self.btn_delete.pack(side="left")
        self._books_buttons = [self.btn_books_all, self.btn_books_isbn,
self.btn_books_fmt, self.btn_books_autor, self.btn_create, self.btn_update,
self.btn_delete]

    # ---------- Config tab ----------
    def _build_config_tab(self):
        f = self.tab_config
        cf = ttk.LabelFrame(f, text="Conexión"); cf.pack(fill="x", padx=10, pady=10)
        self.ip_var = tk.StringVar(value=self.cfg["ip"]); self.port_var =
tk.StringVar(value=str(self.cfg["port"]))
        row = ttk.Frame(cf); row.pack(fill="x", pady=4)
        ttk.Label(row, text="IP/Host:").pack(side="left"); ttk.Entry(row,
textvariable=self.ip_var, width=25).pack(side="left", padx=4)
        row = ttk.Frame(cf); row.pack(fill="x", pady=4)
        ttk.Label(row, text="Puerto:").pack(side="left"); ttk.Entry(row,
textvariable=self.port_var, width=10).pack(side="left", padx=4)
        ttk.Button(cf, text="Guardar conexión",
command=self._save_connection).pack(side="left", padx=10)

        ef = ttk.LabelFrame(f, text="Endpoints"); ef.pack(fill="x", padx=10, pady=10)
        self.ep_vars = {}
        for k, v in self.cfg["endpoints"].items():
            r = ttk.Frame(ef); r.pack(fill="x", pady=2)
            ttk.Label(r, text=k, width=18).pack(side="left")
            var = tk.StringVar(value=v); self.ep_vars[k] = var
            ttk.Entry(r, textvariable=var, width=60).pack(side="left", padx=4)
```

```
        ttk.Button(ef, text="Guardar endpoints",
command=self._save_endpoints).pack(side="left", padx=10, pady=6)

        tf = ttk.LabelFrame(f, text="Tokens (pegar/actualizar manual)")
        tf.pack(fill="x", padx=10, pady=10)
        self.manual_access = tk.StringVar(value=self.cfg.get("access_token",""))
        self.manual_refresh = tk.StringVar(value=self.cfg.get("refresh_token",""))
        r = ttk.Frame(tf); r.pack(fill="x", pady=3)
        ttk.Label(r, text="Access:").pack(side="left"); ttk.Entry(r,
textvariable=self.manual_access, width=90).pack(side="left", padx=4)
        r = ttk.Frame(tf); r.pack(fill="x", pady=3)
        ttk.Label(r, text="Refresh:").pack(side="left"); ttk.Entry(r,
textvariable=self.manual_refresh, width=90).pack(side="left", padx=4)
        ttk.Button(tf, text="Guardar tokens",
command=self._save_tokens_manual).pack(side="left", padx=10, pady=6)

    # ---------- Health ----------
    def _start_health_monitor(self):
        def loop():
            ok = self.client.ping()
            self._set_semaforo("verde" if ok else "rojo")
            self.after(5000, self._start_health_monitor)
        threading.Thread(target=loop, daemon=True).start()

    def _set_semaforo(self, color):
        fill = {"verde": "#2e8b57", "naranja": "#ff8c00", "rojo":
"#b22222"}.get(color, "#b22222")
        try: self.sem_canvas.itemconfig(self.sem_light, fill=fill)
        except tk.TclError: pass

    # ---------- Log helpers ----------
    def _now_ts(self):
        return time.strftime("%H:%M:%S")

    def _passes_level(self, level: str) -> bool:
        order = {"DEBUG":0,"INFO":1,"WARN":2,"ERROR":3}
        want  = order.get(self.verbose_level.get(), 1)
        have  = order.get(level, 1)
        return have >= want

    def _log_line(self, level: str, tag: str, msg: str):
        # filtro por nivel
        if not self._passes_level(level):
            return
        ts = f"[{self._now_ts()}] "
        line = f"{ts}{level:<5} {tag:<7} {msg}"
        # a archivo
        _append_file_log(line)
        # a UI (thread-safe)
        def write():
            self.log_text.insert("end", ts, ("TS",))
            self.log_text.insert("end", f"{level:<5} ", (level,))
            self.log_text.insert("end", f"{tag:<7} ", (tag if tag in
["HTTP","JWT","BOOKS","HEALTH","CFG"] else "INFO",))
```

```
            self.log_text.insert("end", msg + "\n")
            self.log_text.see("end")
        self.after(0, write)

    def _log_http_detail(self, method, url, req_headers, req_body, resp:
requests.Response):
        # REQUEST
        self._log_line("INFO", "HTTP", f">>> {method} {url}")
        if self.show_headers.get() and req_headers:
            for k, v in req_headers.items():
                self._log_line("DEBUG", "HEAD", f"{k}: {v}")
        if self.show_body.get() and req_body is not None:
            try:
                body_str = json.dumps(req_body, ensure_ascii=False)
            except Exception:
                body_str = str(req_body)
            self._log_line("DEBUG", "BODY", body_str)

        # RESPONSE
        status = getattr(resp, "status_code", "N/A")
        self._log_line("INFO", "HTTP", f"<<< {status} {resp.reason if
hasattr(resp,'reason') else ''}".strip())
        if self.show_headers.get():
            try:
                for k, v in resp.headers.items():
                    self._log_line("DEBUG", "HEAD", f"{k}: {v}")
            except Exception:
                pass
        if self.show_body.get():
            preview = (resp.text or "")
            self._log_line("DEBUG", "BODY", preview if len(preview) < 5000 else
preview[:5000] + " ...[truncado]")

    def _clear_log(self):
        self.log_text.delete("1.0", "end")

    def _save_log_as(self):
        try:
            content = self.log_text.get("1.0", "end")
            path = filedialog.asksaveasfilename(
                defaultextension=".log",
                filetypes=[("Log", "*.log"), ("Texto", "*.txt"), ("Todos", "*.*")],
                initialfile=f"jwt_books_{int(time.time())}.log",
                title="Guardar log como…",
            )
            if path:
                with open(path, "w", encoding="utf-8") as f:
                    f.write(content)
                messagebox.showinfo("Log", f"Log guardado en:\n{path}")
        except Exception as e:
            messagebox.showerror("Log", f"No pude guardar el log: {e}")

    def _copy_curl(self):
        req = self.client.last_request
```

```
        if not req:
            messagebox.showwarning("cURL", "Aún no hay requests capturados.")
            return
        method = req.get("method","GET")
        url    = req.get("url","")
        headers= req.get("headers",{}) or {}
        body   = req.get("body", None)

        parts = [f"curl -s -X {method} {self._shell_quote(url)}"]
        for k,v in headers.items():
            parts.append(f"-H {self._shell_quote(f'{k}: {v}')}")
        if body is not None:
            try:
                if isinstance(body, (dict,list)):
                    body_str = json.dumps(body, ensure_ascii=False)
                else:
                    body_str = str(body)
            except Exception:
                body_str = str(body)
            parts.append(f"--data {self._shell_quote(body_str)}")
        cmd = " \\\n  ".join(parts)
        self.clipboard_clear()
        self.clipboard_append(cmd)
        self._log_line("INFO", "HTTP", "cURL copiado al portapapeles")

    def _shell_quote(self, s: str) -> str:
        # comillas dobles con escapes básicos para Windows y *nix
        return '"' + s.replace('"', '\\"') + '"'

    # ---------- Utils ----------
    def _run_bg(self, target):
        threading.Thread(target=target, daemon=True).start()

    def _pretty_xml(self, xml_str):
        try:
            from xml.dom import minidom
            parsed = minidom.parseString(xml_str.encode("utf-8") if
isinstance(xml_str, str) else xml_str)
            return parsed.toprettyxml(indent="  ")
        except Exception:
            return xml_str

    def _update_token_fields(self):
        at = self.cfg.get("access_token",""); rt = self.cfg.get("refresh_token","")
        self.at_var.set((at[:80] + "...") if at else "")
        self.rt_var.set((rt[:80] + "...") if rt else "")

    def _set_books_enabled(self, enabled: bool):
        state = "normal" if enabled else "disabled"
        for btn in getattr(self, "_books_buttons", []):
            try: btn.config(state=state)
            except: pass

    # ---------- Actions ----------
```

```
    def _ping_now(self):
        self._set_semaforo("naranja")
        def run():
            ok = self.client.ping()
            self._set_semaforo("verde" if ok else "rojo")
        self._run_bg(run)

    def _do_register(self):
        u = self.reg_user.get().strip(); p = self.reg_pass.get().strip()
        if not u or not p:
            messagebox.showwarning("Faltan datos", "Usuario y contraseña son
requeridos")
            return
        def run():
            resp = self.client.register(u, p)
            self._set_semaforo("verde" if resp.ok else "rojo")
        self._run_bg(run)

    def _do_login(self):
        u = self.login_user.get().strip(); p = self.login_pass.get().strip()
        if not u or not p:
            messagebox.showwarning("Faltan datos", "Usuario y contraseña son
requeridos")
            return
        def run():
            self._set_semaforo("naranja")
            resp = self.client.login(u, p)
            def ui_after_login():
                self._update_token_fields()
                have_token = bool(self.cfg.get("access_token"))
                self.auth_ready = have_token
                self._set_books_enabled(have_token)
                if have_token:
                    self._log_tokens()
            self.after(0, ui_after_login)
            self._set_semaforo("verde" if resp.ok else "rojo")
        self._run_bg(run)

    def _do_refresh(self):
        def run():
            self._set_semaforo("naranja")
            ok = self.client.refresh_access_token()
            def ui_after_refresh():
                self._update_token_fields()
                have_token = bool(self.cfg.get("access_token"))
                self.auth_ready = have_token
                self._set_books_enabled(have_token)
            self.after(0, ui_after_refresh)
            self._set_semaforo("verde" if ok else "rojo")
        self._run_bg(run)

    def _do_logout(self):
        def run():
            self._set_semaforo("naranja")
```

```
            resp = self.client.logout()
            def ui_after_logout():
                self._update_token_fields()
                self.auth_ready = False
                self._set_books_enabled(False)
                self._render_books([])
            self.after(0, ui_after_logout)
            self._set_semaforo("verde" if resp.ok else "rojo")
        self._run_bg(run)

    def _log_tokens(self):
        self._log_line("INFO", "JWT", "Decodificar y loguear JWT (iniciado)")
        try:
            at = self.cfg.get("access_token", "") or ""
            rt = self.cfg.get("refresh_token", "") or ""
            if not at and not rt:
                self._log_line("WARN", "JWT", "No hay tokens en memoria (haz Login
primero)")
                messagebox.showwarning("JWT", "No hay tokens cargados. Realiza Login
primero.")
                return
            self.client.log_tokens()
            at_short = (at[:32] + "...") if at else "<vacío>"
            rt_short = (rt[:32] + "...") if rt else "<vacío>"
            messagebox.showinfo("JWT decodificado",
                                f"ACCESS: {at_short}\nREFRESH: {rt_short}\n\nRevisa el
Log para header/payload/exp.")
        except Exception as e:
            self._log_line("ERROR", "JWT", f"Error al decodificar: {e}")
            messagebox.showerror("JWT", f"Error al decodificar: {e}")
        finally:
            self._log_line("INFO", "JWT", "Decodificar y loguear JWT (terminado)")

    # ----- Books actions -----
    def _render_books(self, items):
        for it in self.tree.get_children(): self.tree.delete(it)
        for row in items:
            self.tree.insert("", "end", values=(row["isbn"], row["title"],
row["authors"], row["year"], row["genre"], row["price"], row["stock"], row["format"]))
        self._log_line("INFO", "BOOKS", f"Renderizadas {len(items)} filas")

    def _ensure_token_or_warn(self) -> bool:
        if not self.cfg.get("access_token"):
            self._log_line("WARN", "BOOKS", "Sin access_token. Haz Login primero.")
            messagebox.showwarning("Libros", "No hay access_token. Realiza Login
primero.")
            return False
        return True

    def _parse_books_xml_to_items(self, xml_text: str):
        root = ET.fromstring(xml_text)
        items = []
        for b in root.findall(".//book"):
            items.append({
```

```
                "isbn": b.get("isbn",""),
                "title": (b.findtext("title") or "").strip(),
                "authors": ", ".join([a.text.strip() for a in b.findall("author") if
a.text]) or "",
                "year": (b.findtext("year") or "").strip(),
                "genre": (b.findtext("genre") or "").strip(),
                "price": (b.findtext("price") or "").strip(),
                "stock": (b.findtext("stock") or "").strip(),
                "format": (b.findtext("format") or "").strip(),
            })
        return items

    def _handle_books_response(self, resp, tag: str, dump_file: str):
        body = resp.text or ""
        # Guardar body de respuesta para auditoría
        try:
            with open(dump_file, "w", encoding="utf-8") as f: f.write(body)
            self._log_line("DEBUG", tag, f"Body guardado en ./{dump_file}")
        except Exception as e:
            self._log_line("WARN", tag, f"No pude guardar body: {e}")

        # Parse + render tabla
        try:
            items = self._parse_books_xml_to_items(body)
            self._log_line("INFO", tag, f"Encontrados {len(items)} libros")
            self.after(0, lambda: self._render_books(items))
        except Exception as e:
            self._log_line("ERROR", tag, f"No es XML válido o estructura distinta:
{e}")
            self._log_line("DEBUG", tag, self._pretty_xml(body))

        self._set_semaforo("verde" if resp.ok else "rojo")

    def _books_all(self):
        if not self._ensure_token_or_warn(): return
        def run():
            self._set_semaforo("naranja")
            resp = self.client.books_all()
            self._handle_books_response(resp, "BOOKS", "last_books_all.xml")
        self._run_bg(run)

    def _books_by_isbn(self):
        if not self._ensure_token_or_warn(): return
        isbn = self.q_isbn.get().strip()
        if not isbn:
            messagebox.showwarning("Faltan datos", "ISBN requerido"); return
        def run():
            self._set_semaforo("naranja")
            resp = self.client.books_by_isbn(isbn)
            self._handle_books_response(resp, "BOOKS/ISBN", "last_books_by_isbn.xml")
        self._run_bg(run)

    def _books_by_format(self):
        if not self._ensure_token_or_warn(): return
```

```
        fmt = self.q_fmt.get().strip()
        if not fmt:
            messagebox.showwarning("Faltan datos", "Format requerido"); return
        def run():
            self._set_semaforo("naranja")
            resp = self.client.books_by_format(fmt)
            self._handle_books_response(resp, "BOOKS/FORMAT",
"last_books_by_format.xml")
        self._run_bg(run)

    def _books_by_autor(self):
        if not self._ensure_token_or_warn(): return
        name = self.q_autor.get().strip()
        if not name:
            messagebox.showwarning("Faltan datos", "Autor (name) requerido"); return
        def run():
            self._set_semaforo("naranja")
            resp = self.client.books_by_autor(name)
            self._handle_books_response(resp, "BOOKS/AUTOR",
"last_books_by_autor.xml")
        self._run_bg(run)

    def _collect_book_payload(self, require_all_for_create=False):
        data = {}
        def set_if(var, key):
            val = var.get().strip()
            if val != "": data[key] = val
        set_if(self.b_isbn, "isbn"); set_if(self.b_titulo, "titulo")
        set_if(self.b_id_autor, "id_autor"); set_if(self.b_id_categoria,
"id_categoria")
        set_if(self.b_id_editorial, "id_editorial"); set_if(self.b_anio,
"anio_publicacion")
        set_if(self.b_price, "price"); set_if(self.b_stock, "stock");
set_if(self.b_formato, "formato")
        if require_all_for_create:
            req =
['isbn','titulo','id_autor','id_categoria','id_editorial','anio_publicacion','price','
stock','formato']
            missing = [k for k in req if k not in data]
            if missing:
                messagebox.showwarning("Faltan campos", f"Para crear faltan: {',
'.join(missing)}")
                return None
        return data

    def _book_create(self):
        if not self._ensure_token_or_warn(): return
        payload = self._collect_book_payload(require_all_for_create=True)
        if payload is None: return
        def run():
            self._set_semaforo("naranja")
            self._log_line("DEBUG", "BOOKS/CREATE", f"payload={payload}")
            resp = self.client.book_create(payload)
            self._set_semaforo("verde" if resp.ok else "rojo")
```

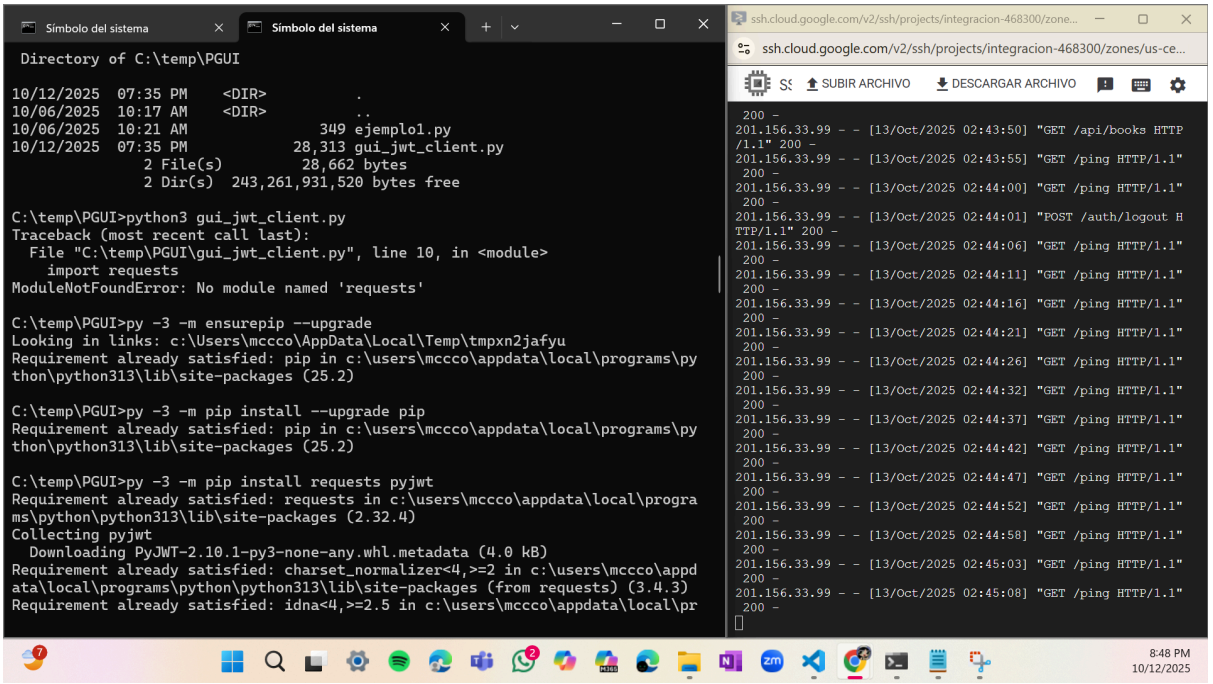
```
        self._run_bg(run)

    def _book_update(self):
        if not self._ensure_token_or_warn(): return
        payload = self._collect_book_payload(require_all_for_create=False)
        if "isbn" not in payload:
            messagebox.showwarning("Falta ISBN", "Para actualizar se requiere
'isbn'"); return
        def run():
            self._set_semaforo("naranja")
            self._log_line("DEBUG", "BOOKS/UPDATE", f"payload={payload}")
            resp = self.client.book_update(payload)
            self._set_semaforo("verde" if resp.ok else "rojo")
        self._run_bg(run)

    def _book_delete(self):
        if not self._ensure_token_or_warn(): return
        isbn = self.b_isbn.get().strip()
        if not isbn:
            messagebox.showwarning("Falta ISBN", "Para borrar se requiere 'isbn'");
return
        def run():
            self._set_semaforo("naranja")
            self._log_line("DEBUG", "BOOKS/DELETE", f"isbn={isbn}")
            resp = self.client.book_delete(isbn)
            self._set_semaforo("verde" if resp.ok else "rojo")
        self._run_bg(run)

    # ----- Config -----
    def _save_connection(self):
        ip = self.ip_var.get().strip(); port_s = self.port_var.get().strip()
        if not ip or not port_s.isdigit():
            messagebox.showwarning("Datos inválidos", "IP y puerto deben ser
válidos"); return
        self.cfg["ip"] = ip; self.cfg["port"] = int(port_s)
        if save_config(self.cfg):
            self.base_url_var.set(self.client.base_url())
            self._log_line("INFO", "CFG", "Conexión guardada")
        else:
            self._log_line("ERROR", "CFG", "Error al guardar conexión")

    def _save_endpoints(self):
        for k, var in self.ep_vars.items():
            self.cfg["endpoints"][k] = var.get().strip() or
DEFAULT_CFG["endpoints"][k]
        if save_config(self.cfg):
            self._log_line("INFO", "CFG", "Endpoints guardados")
        else:
            self._log_line("ERROR", "CFG", "Error al guardar endpoints")

    def _save_tokens_manual(self):
        self.cfg["access_token"] = self.manual_access.get().strip()
        self.cfg["refresh_token"] = self.manual_refresh.get().strip()
        ok = save_config(self.cfg)
```

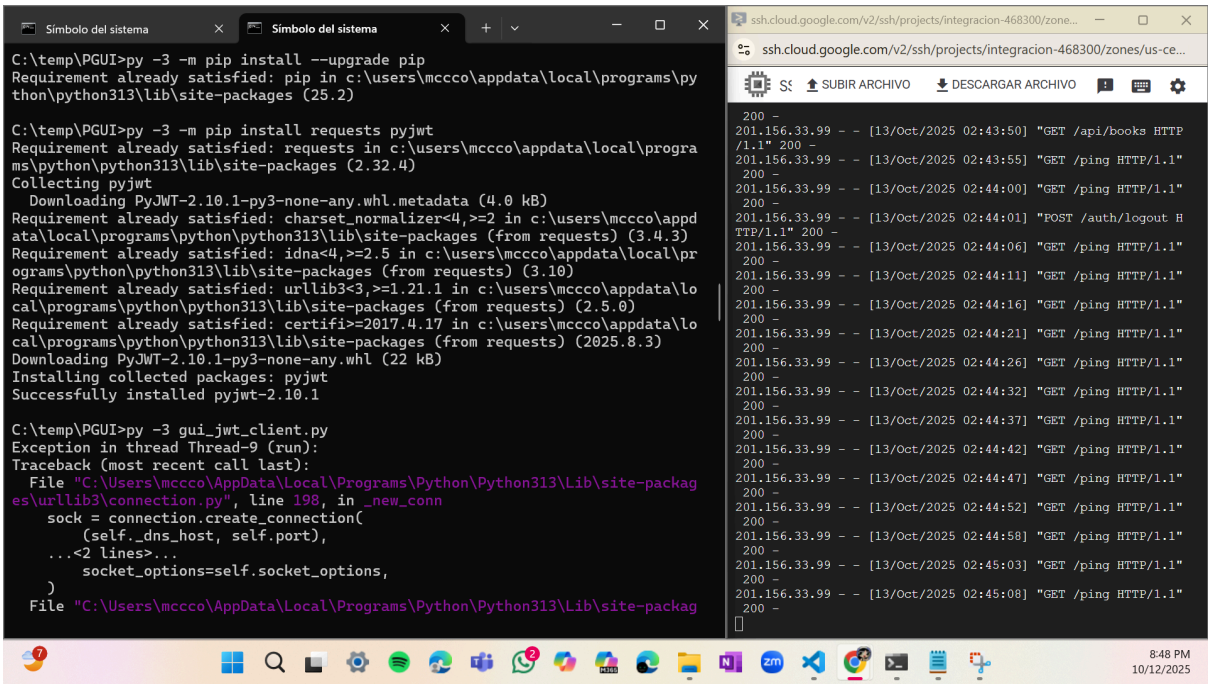
```
        self._update_token_fields()
        self._set_books_enabled(bool(self.cfg.get("access_token")))
        self._log_line("INFO", "CFG", "Tokens guardados" if ok else "Error al guardar
tokens")


if __name__ == "__main__":
    app = App()
    # Trae al frente por si quedó fuera de pantalla
    def _bring_to_front():
        try:
            app.geometry("+50+50"); app.deiconify(); app.lift(); app.focus_force()
            app.attributes("-topmost", True); app.after(300, lambda:
app.attributes("-topmost", False))
        except Exception:
            pass
    app.after(100, _bring_to_front)
    app.mainloop()
```

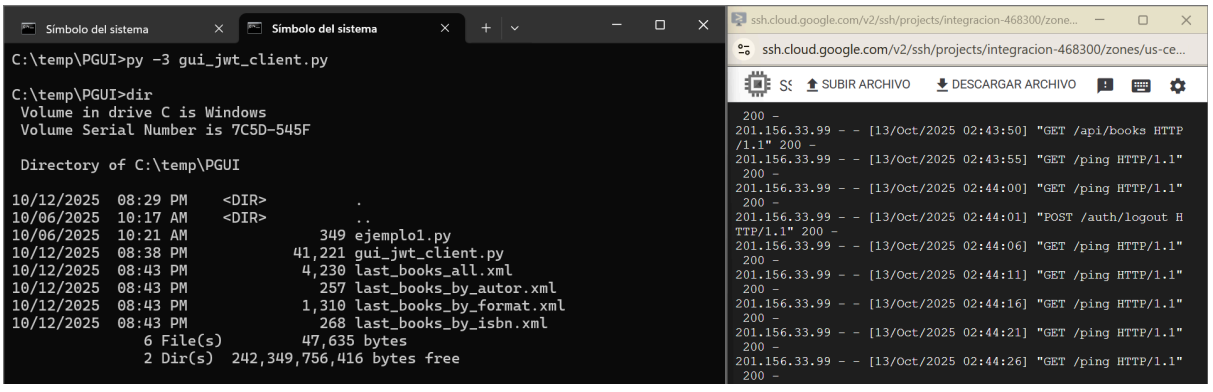
Tras guardar este codigo con el commando code **gui_jwt_client.py,** la IA me dijo que antes de poder correr este en el CMD tuve que instalar las siguientes dependencias en el subdirectorio *PGUI:*

```
Python
py -3 -m ensurepip --upgrade
py -3 -m pip install --upgrade pip
py -3 -m pip install requests pyjwt
py -3 gui_jwt_client.py
```

Este último comando, **py -3 gui_jwt_client.py**, es el que ejecuta el programa y abre la ventana de TKinter.







El video de funcionamiento del programa en TKinter es el siguiente:

https://youtu.be/1jmjtQA3pSQ

Conclusión

Utilizar Tkinter como interfaz gráfica en lugar de desarrollar una aplicación web tradicional resultó ser una alternativa práctica y eficiente, especialmente en entornos controlados o educativos. La interfaz permite autenticarse mediante tokens JWT, registrar usuarios, gestionar libros y monitorear el estado del microservicio, todo desde una aplicación de escritorio desarrollada completamente en Python. Aunque no ofrece la escalabilidad ni la adaptabilidad de una interfaz web moderna, su implementación fue más directa y no requirió aprender múltiples tecnologías. Esta experiencia me demuestra que, en ciertos contextos, una aplicación de escritorio con Tkinter puede ser una solución suficientemente robusta para interactuar con microservicios actuales.